Corrección del primer examen parcial

Medina Reyes Miguel Angel

1.- ¿Qué representa un atributo y que representa un metodo? Da un ejemplo de cada uno.

Corrección. Los conceptos están invertidos. lo que puede hacer elobjeto es un método, no un atributo, y un metodo no es el objeto. Un atributo guarda parte del estado (por ejemplo, la velocidad de un coche) y un metodo es un comportamiento (por ejemplo, acelerar).

2.- ¿Que hace el metodo main y por qué se declara istatic en los programas que usamos en la clase?

Corrección. Es correcto que main es el punto desde el que inicia el programa. La explicación de static es incorrecta: se declara static para que la JVM pueda invocarlo desde la clase sin crear antes una instancia.

3.- ¿Qué diferencia hay entre un atributo de instancia y uno static? Explica que comparten dos objetos de la misma clase.

Corrección. La descripción del atributo es incorrecta: no desaparece al terminar una linea, sino que vive mientras exista su objeto. Es valido que el static dura todo el programa, pero falta la idea central: cada objeto tiene su propia copia del atributo de instancia, y el static pertenece a la clase y todos los objetos lo comparten, como un contador de objetos creados.

4.- ¿Qué guarda una variable de tipo primitivo y qué guarda una de referencia? Da un ejemplo de cada una.

Corrección. Es valido que la variable de referencia guarda la direccion (la referencia) de un objeto, En la primitiva no se dice que se guarda el mismo valor, y faltan los ejemplos: int guarda directamente un 8 y una variable String guarda la referencia al objeto de texto.

FIRST CLASS

Escaneado con CamScanner

5- ¿Qué hace javac con un archivo .java y que hace JVM con el resultado

Corrección. Es correcto que javac generá el .class a partir del .java, pero javac no ejecuta nada: compila el código fuente a bytecode. Falta la segunda parte: la JVM ejecuta ese bytecode

¿Qué imprime?

Corrección. Las lineas 3, 2 y 2.0 son correctas. Falta la tercera linea: el while sí llega a i=2 e imprime 3/2=1 En la cuarta linea se sumaron 7 y 9 para dar 16, cuando debian concatenarse "B" + 7 + 9 = B79; los valores 7 y 9 sí están bien identificados. La salida exacta era 3/2/1/B79 y 2.0

La linea de código del while su salida ocurre gracias a que compara la 'i' tiene que ser igual o mayor a tres para detenerse y como esta inicializado en cero y tambien se utiliza en en contador aumenta 1 en su primera vuelta, 2, en la segunda y 3 en su ultima vuelta.

Vuelta	Resultado
0	3
1	2
2	1
3	Se detiene

En el segundo caso, se imprime B79, porque los valores 7 y 9 estan concatenados, no se estan sumando y B aparece como una letra impresa por el println.

Por ultimo el ultimo codigo de impresión muestra ese valor porque primero hace la operacion (datos[1]/2); y despues a ese valor lo convierte en un double para que se almacene en la variable double mitad

FIRST CLASS

Escaneado con CamScanner

¿Compila, truena o funciona?

a) Corrección. La clasificación es incorrecta: el código funciona y no truena. La regla correcta es: sin break la ejecución sigue al siguiente case. Falto la salida:

Si compila, el error esta en la falta de break, lo que provoca que el programa continue ejecutando las demas sentencias hasta que ya no tenga ninguna sentencia por ejecutar (llegar al final del switch) o encuentre un return, a esta acción se le conoce como "fall-through".

b) Corrección. Correcto: no compila, por que el nivel solo existe dentro de las llaves donde se declara. Tecnicamente es un error de ambito (simbolo no encontrado) y no de sintaxis.

Las {} hacen que el int viva solo dentro de ellas provocando el error.

c) Corrección. Incorrecto: Compila pero truena. Llamar trim() sobre una referencia null lanza NullPointerException, y no se imprime nada (tampoco "null").

El error esta en que tratamos de llamar y ejecutar el metodo con .trim() (Que sirve para eliminar espacios en blanco) sobre esa variable y como etiqueta es igual a null Java no tiene una cadena de texto real sobre la cual ejecutar el método.

FIRST CLASS

Escaneado con CamScanner

Modelado: base, reto y justificación

```
public class Cargador{
    final int capacidad;
    int ocupadas;
    static int fabricados = 0;

    public Cargador (int capacidad){
        this.capacidad = capacidad;
        this.ocupadas = 0;
        fabricados ++;
    }

    public void colocar (){
        ocupadas ++;
    }

    public int getOcupadas (){
        return ocupadas;
    }
}
```

Justificación

Se utiliza int para representar cantidades numericas enteras

Se utiliza las Instancias / Clase capacidad y ocupadas son de instancias que pertenecen a cada objeto individual

Mutables y final: capacidad es ~~fi~~nable (inmutable tras construirse) mientras que ocupadas y fabricados son mutables

*Constructor: Inicializa la capacidad especifica, fija las ocupadas en 0 (nace vacio) e incrementa el contador ~~final~~ global de fabricados

*Métodos: colocar() es un metodo de cambio que muta el estado incrementando una unidad; getOcupadas() es de consulta y retorna el valor actual.

FIRST CLASS

Escaneado con CamScanner

Codigo con Repeticion

```
static int intentoValido(int[] intentos, int clave){
    int i=0;
    do {
        if (intentos[i]==clave){
            return i+1;
        }
        i++;
    } while (i<3);
return 0;
}
```

Correccion. Falta la cabecera static intentoValido(int[] intentos, int clave) y el arreglo no se recorre: lee del teclado e imprime, cosa que el enunciado prohibe. Se reconocen el uso de do-while y la idea de comparar un intento con la clave. La condicion while (clave == contraseña) esta invertida y no se detiene al acertar, tampoco hay numeracion desde 1 ni return 0 al fracasar. La idea esperada era un indice i dentro de un do while, que en cada vuelta revise si intentos[i] == clave para hacer un return i+1, avance ministerio <3 y al final haya return 0.

Escaneado con CamScanner

Codigo con selección

Corrección. Bien: revisa bateria < 10 antes del switch, lo que le da prioridad incluso con null, y usa break en cada clase. Las cuatro acciones estan en el orden correcto, pero los case usan 1-4 en lugar de APAGADO, MEDIR, CALIBRAR, y PRUEBA, y ademas escribe "Simular" en vez de "SIMULAR". Tambien falla en esto: no hay cabecera no lee del teclado e imprime en vez de devolver con return, escribe "CARGAR BATERIA" en vez de "CARGAR" y no devuelve "SIN MODO" cuando modo es null y la bateria alcanza. En la sintaxis: printl, Syten y default sin dos puntos.

```
static String accion (Modo modo, int bateria) {
        if (bateria < 10) {
            return "CARGAR";
        }
        if (modo == null) {
            return "SIN MODO";
        }
        Switch (modo) {
            case APAGADO:
                return "DETENER";
            case MEDIR:
                return "LEER";
            case CALIBRAR:
                return "AJUSTAR";
            case PRUEBA:
                return "SIMULAR";
            default:
                return "";
        }
}
```

FIRST CLASS

Escaneado con CamScanner